https://www.maxicours.com/se/cours/rechercher-et-inserer-une-cle-dans-un-arbre-binaire-de-recherche/

Rechercher et insérer une clé dans un arbre binaire de recherche

Lycée > Terminale > NSI > Rechercher et insérer une clé dans un arbre binaire de recherche

Objectifs

- Connaitre l'algorithme de recherche d'une clé dans un arbre binaire de recherche et savoir le programmer en Python.
- Connaitre l'algorithme d'insertion d'une clé dans un arbre binaire de recherche et savoir le programmer en Python.

Points clés

- La recherche d'une clé dans un arbre binaire de recherche consiste à comparer les nœuds et la clé et, à chaque fois, à rechercher dans le sous-arbre gauche ou le sous-arbre droit, selon que la clé est plus petite ou plus grande que le nœud.
- L'insertion d'une clé dans un arbre binaire de recherche consiste à parcourir l'arbre et à placer la clé là où la recherche s'est arrêtée.

Pour bien comprendre

- Comprendre la structure d'un arbre binaire de recherche.
- Utiliser Python dans les arbres binaires de recherche.

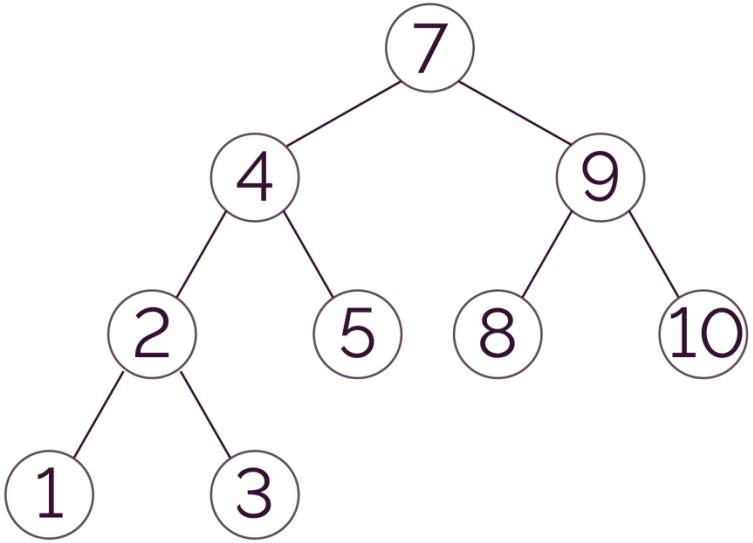
1. Rechercher une clé dans un arbre binaire de recherche

a. Principe
La **recherche d'une clé** dans un arbre binaire de recherche non vide consiste à **comparer la clé** et la racine, puis, si elles sont différentes, à descendre dans le sous-arbre gauche ou le sous-arbre droit selon que la clé est plus petite ou plus grande que la racine.

La recherche s'arrête quand on trouve la clé dans l'arbre ou quand on a atteint une feuille.

- Si la clé est inférieure à la racine de l'arbre, on recherche dans le sous-arbre gauche.
- Si la clé est supérieure à la racine de l'arbre, on recherche dans le sous-arbre droit.

Exemple
On considère l’arbre binaire de recherche suivant.



On recherche la clé **8** dans cet arbre.

- On compare la racine et la clé : 7 < **8**.
  Donc on cherche dans le sous-arbre droit.
- On compare la racine du sous-arbre droit et la clé : 9 > **8**.
  Donc on cherche dans son sous-arbre gauche.
- On compare la racine du sous-arbre gauche du sous-arbre droit et la clé : 8 = **8**. La clé **8** est trouvée !

On recherche maintenant la clé **6** dans cet arbre.

- On compare la racine et la clé : 7 > **6**.
  Donc on cherche dans le sous-arbre gauche.
- On compare la racine du sous-arbre gauche et la clé : 4 < **6**.
  Donc on cherche dans son sous-arbre droit.
- On compare la racine du sous-arbre droit du sous-arbre gauche et la clé : 5 < **6**.
  Or 5 est une feuille. La clé **6** n’est donc pas dans l’arbre.

b. Algorithme

Voici ci-dessous un algorithme de la fonction récursive de recherche d'une clé dans un arbre binaire de recherche a.

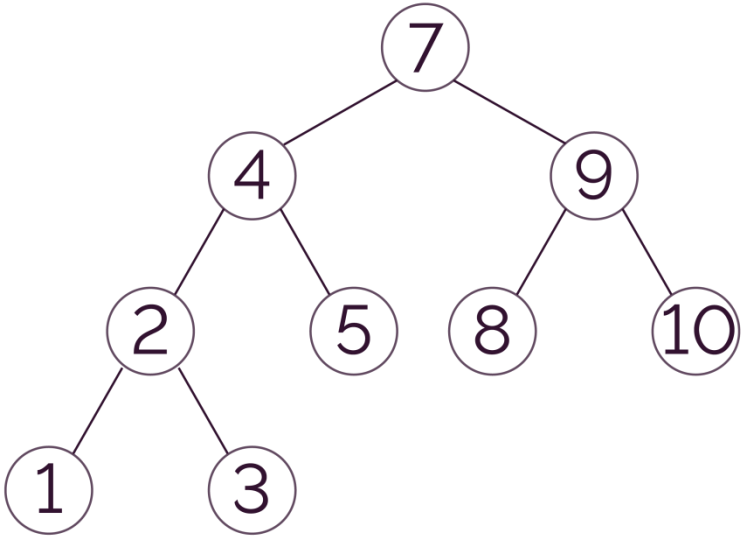
Algorithme	Explication
Fonction **recherche**(ABR a, cle):	On définit la fonction recherche qui prend pour paramètres un arbre binaire de recherche a de type ABR et une clé cle.
Si est_vide(a), alors retourner Faux	Si a est vide, alors on retourne Faux.
Autrement si racine(a) = cle, alors retourner Vrai	Si la cle est égale à la racine de a, alors on retourne Vrai.
Sinon	Sinon
Si racine(a) < cle , alors retourner **recherche**(sad(a), cle)	si la racine de a est inférieure à la cle, on recherche dans le sous-arbre droit de a.
Sinon, retourner **recherche**(sag(a), cle)	Sinon on recherche dans le sous-arbre gauche de a.
FinSi	Fin de l'instruction conditionnelle.
FinSi	Fin de l'instruction conditionnelle.

c. Programmation

En Python, la fonction recherche(arbre, cle) implémente la recherche de l'élément cle dans l'arbre binaire de recherche arbre.

Python	Explication
def recherche(arbre, cle):	On définit la fonction recherche.
if arbre.est_vide(): return False	Si arbre est vide, on retourne False.
elif arbre.etiquette == cle: return True	Autrement, si l'étiquette de la racine de l'arbre arbre est égale à cle, on retourne True.
else:	Sinon
if arbre.etiquette < cle: return recherche(arbre.sad, cle)	si l'étiquette de arbre est inférieure à cle, on retourne recherche (arbre.sad, cle).
else: return recherche(arbre.sag, cle)	Sinon on retourne recherche (arbre.sag, cle).

Exemple
On considère à nouveau l'arbre binaire de recherche suivant.



Voici la fonction précédente testée avec cet arbre sur Python Tutor :

Python 3.6

```
1   class ArbreBinaire:
2       def __init__(self, etiquette = None, sag = None, sad = None):
3           self.etiquette = etiquette
4           self.sag = sag
5           self.sad = sad
6
7       def est_vide(self):
8           if self.etiquette is None:
9               return True
10          else:
11              return False
12
13  # Définition de l'arbre :
```

```
#       7
#     /   \
#    4     9
#   / \   / \
#  2   5 8   10
# / \
# 1   3

a = ArbreBinaire()
a.etiquette = 7
a.sag = ArbreBinaire(4)
a.sad = ArbreBinaire(9)
a.sag.sag = ArbreBinaire(2)
a.sag.sad = ArbreBinaire(5, ArbreBinaire(), ArbreBinaire())
a.sad.sag = ArbreBinaire(8, ArbreBinaire(), ArbreBinaire())
a.sad.sad = ArbreBinaire(10, ArbreBinaire(), ArbreBinaire())
a.sag.sag.sag = ArbreBinaire(1, ArbreBinaire(), ArbreBinaire())
a.sag.sag.sad = ArbreBinaire(3, ArbreBinaire(), ArbreBinaire())
```

```
32
33
34
35  def recherche(arbre, cle):
36      if arbre.est_vide():
37          return False
38      elif arbre.etiquette == cle:
39          return True
40      else:
41          if arbre.etiquette < cle:
42              return recherche(arbre.sad, cle)
43          else:
44              return recherche(arbre.sag, cle)
45
46
47  print("Recherche de 8 : ", recherche(a, 8))
48  print("Recherche de 6 : ", recherche(a, 6))
```

Ces lignes de code permettent de définir la fonction recherche.

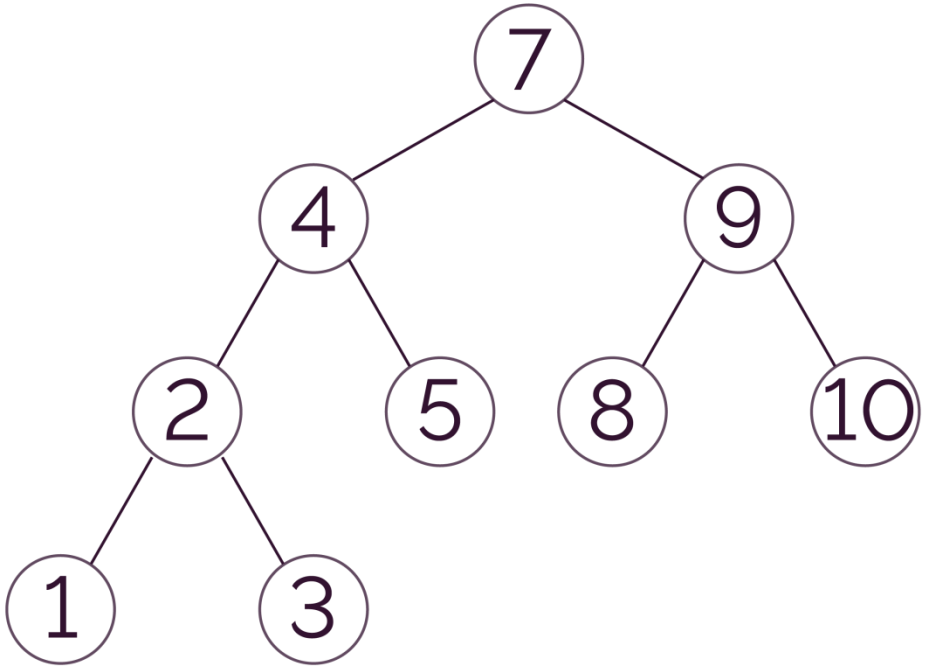
On pourra ainsi l'utiliser sur l'arbre étudié : print(recherche(a, 8)) renverra True, donc 8 est bien dans l'arbre. Également, print(recherche(a, 6)) renverra False, donc 6 n'est pas dans l'arbre.

2. Insérer une clé dans un arbre binaire de recherche

a. Principe
L'**insertion d'une clé** dans un arbre binaire de recherche non vide consiste à parcourir l'arbre comme pour rechercher une clé.

On crée ensuite, avec la **clé**, un nouveau nœud à l'endroit où la recherche s'est arrêtée.

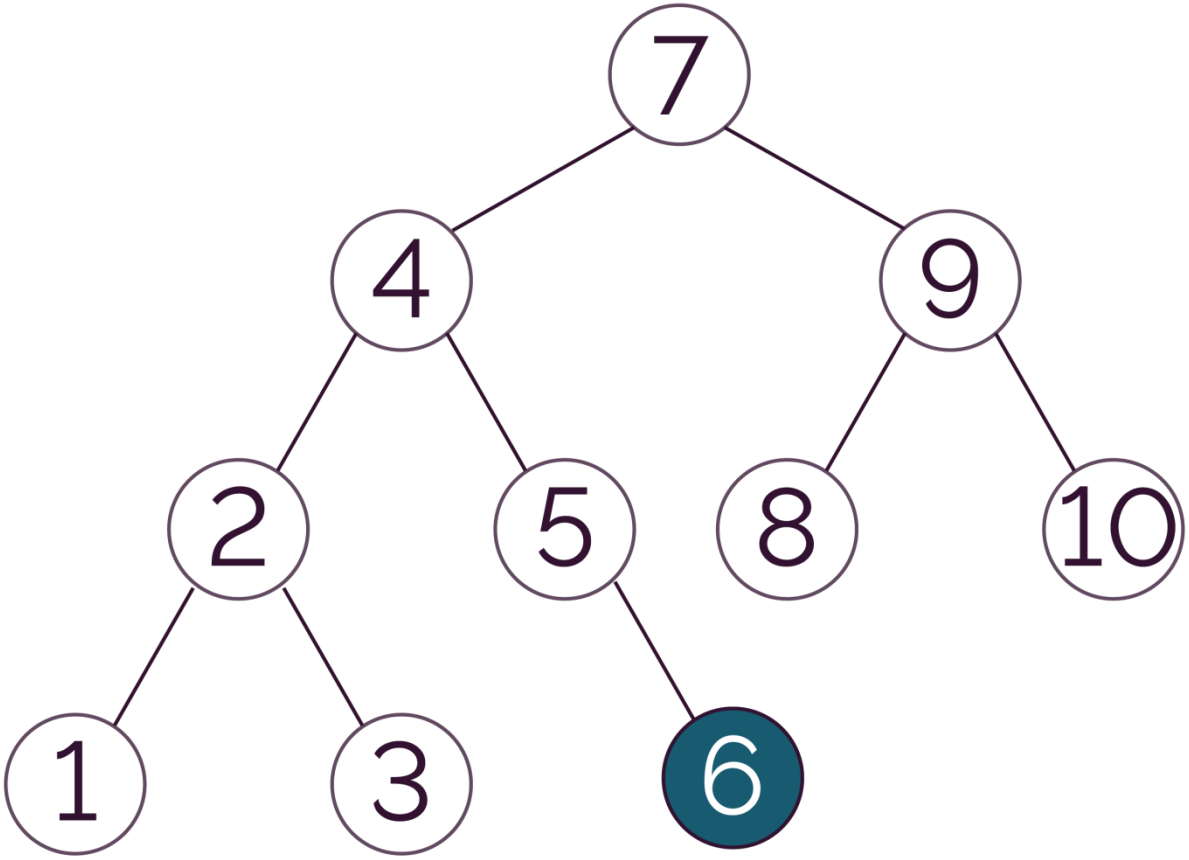
Exemple
On considère l'arbre binaire de recherche suivant.



On veut insérer la clé **6** dans cet arbre, on réalise donc les comparaisons suivantes.

- On compare la racine et la clé : 7 > **6**.
  Donc on parcourt le sous-arbre gauche du nœud 7.
- On compare la racine du sous-arbre gauche et la clé : 4 > **6**.
  Donc on parcourt le sous-arbre droit du nœud 4.
- On compare la racine du sous-arbre droit du nœud 4 et la clé : 5 > **6**.
  Or 5 est une feuille. La clé **6** sera donc l'étiquette du sous-arbre droit du nœud 5.

On obtient ainsi l'arbre suivant.



b. Algorithme

Voici ci-dessous un algorithme de la fonction récursive d'insertion d'une clé dans un arbre binaire de recherche a.

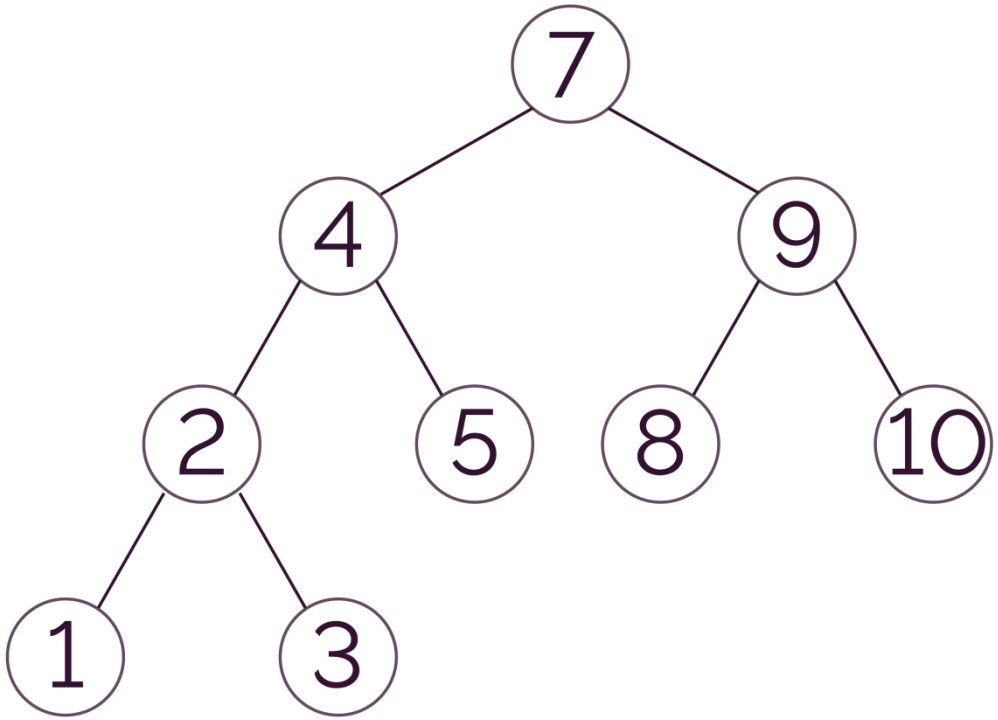
Algorithme	Explication
Fonction insertion(ABR a, cle):	On définit la fonction insertion qui prend pour paramètres un arbre binaire de recherche a de type ABR et une clé cle.
Si est_vide(a), alors racine(a) = cle	Si a est vide, alors cle sera la racine de cle.
Sinon	Sinon
Si racine(a) < cle, alors	si la racine de a est inférieure à la cle,
Si est_vide(sad(a)), alors racine(sad(a)) = cle	si le sous-arbre droit de a est vide, alors cle sera la racine du sous-arbre droit de a.
Sinon, retourner **insertion**(sad(a), cle)	sinon on insère cle dans le sous-arbre droit de a.
FinSi	Fin de l'instruction conditionnelle.
Autrement si racine(a) > cle, alors	si la racine de a est supérieure à la cle,
Si est_vide(sag(a)), alors racine(sag(a)) = cle	si le sous-arbre gauche de a est vide, alors cle sera la racine du sous-arbre gauche de a.
Sinon, retourner **insertion**(sag(a), cle)	sinon on insère cle dans le sous-arbre gauche de a
FinSi	Fin de l'instruction conditionnelle.
Sinon, la clé est déjà dans l'arbre	
FinSi	Fin de l'instruction conditionnelle.
FinSi	Fin de l'instruction conditionnelle.

c. Programmation

En Python, la fonction insertion(arbre, cle) implémente l'insertion de l'élément cle dans l'arbre binaire de recherche arbre.

Python	Explication
def insertion(arbre, cle):	On définit la fonction insertion.
if arbre.est_vide(): arbre.etiquette = cle	Si arbre est vide, on lui attribue cle comme étiquette.
else:	Sinon
if arbre.etiquette < cle:	si l'étiquette de arbre est inférieure à cle,
if arbre.sad.est_vide(): arbre.sad.etiquette = cle	si le sous-arbre droit de arbre est vide, on lui attribue cle comme étiquette.
else: return insertion(arbre.sad, cle)	sinon on insère l'élément cle dans le sous-arbre droit de arbre.
elif arbre.etiquette > cle:	autrement, si l'étiquette de arbre est supérieure à cle,
if arbre.sag.est_vide(): arbre.sag.etiquette = cle	si le sous-arbre gauche de arbre est vide, on lui attribue cle comme étiquette.
else: return insertion(arbre.sag, cle)	sinon on insère l'élément cle dans le sous-arbre gauche de arbre.
else: return "la clé est déjà présente dans l'arbre."	Sinon on retourne le message "la clé est déjà présente dans l'arbre."

Exemple
On considère à nouveau l'arbre binaire de recherche suivant.


7
4
9
2
5
8
10
1
3

Voici la fonction précédente testée avec cet arbre sur Python Tutor.

Python 3.6

```
1   class ArbreBinaire:
2       def __init__(self, etiquette = None, sag = None, sad = None):
3           self.etiquette = etiquette
4           self.sag = sag
5           self.sad = sad
6
7       def est_vide(self):
8           if self.etiquette is None:
9               return True
10          else:
11              return False
12
13  # Définition de l'arbre :
14  #       7
```

```
#     /    \
#    4      9
#   / \    / \
#  2   5  8   10
# / \
# 1   3

a = ArbreBinaire()
a.etiquette = 7
a.sag = ArbreBinaire(4)
a.sad = ArbreBinaire(9)
a.sag.sag = ArbreBinaire(2)
a.sag.sad = ArbreBinaire(5, ArbreBinaire(), ArbreBinaire())
a.sad.sag = ArbreBinaire(8, ArbreBinaire(), ArbreBinaire())
a.sad.sad = ArbreBinaire(10, ArbreBinaire(), ArbreBinaire())
a.sag.sag.sag = ArbreBinaire(1, ArbreBinaire(), ArbreBinaire())
a.sag.sag.sad = ArbreBinaire(3, ArbreBinaire(), ArbreBinaire())
```

```
33
34
35  def insertion(arbre, cle):
36      if arbre.est_vide():
37          arbre.etiquette = cle
38          return "Fait !"
39      else:
40          if arbre.etiquette < cle:
41              if arbre.sad.est_vide():
42                  arbre.sad.etiquette = cle
43                  return "Fait !"
44              else:
45                  return insertion(arbre.sad, cle)
46          elif arbre.etiquette > cle:
47              if arbre.sag.est_vide():
48                  arbre.sag.etiquette = cle
49                  return "Fait !"
50              else:
```

```
            return insertion(arbre.sag, cle)
        else:
            return "Cette clé est déjà présente dans l'arbre."


    print("Insertion de 6 : ", insertion(a, 6))
    print("Vérification de la présence de 6 : ", a.sag.sad.sad.etiquette == 6)
```

a.sag.sad.sad.etiquette renvoie 6. La clé 6 a bien été insérée.